

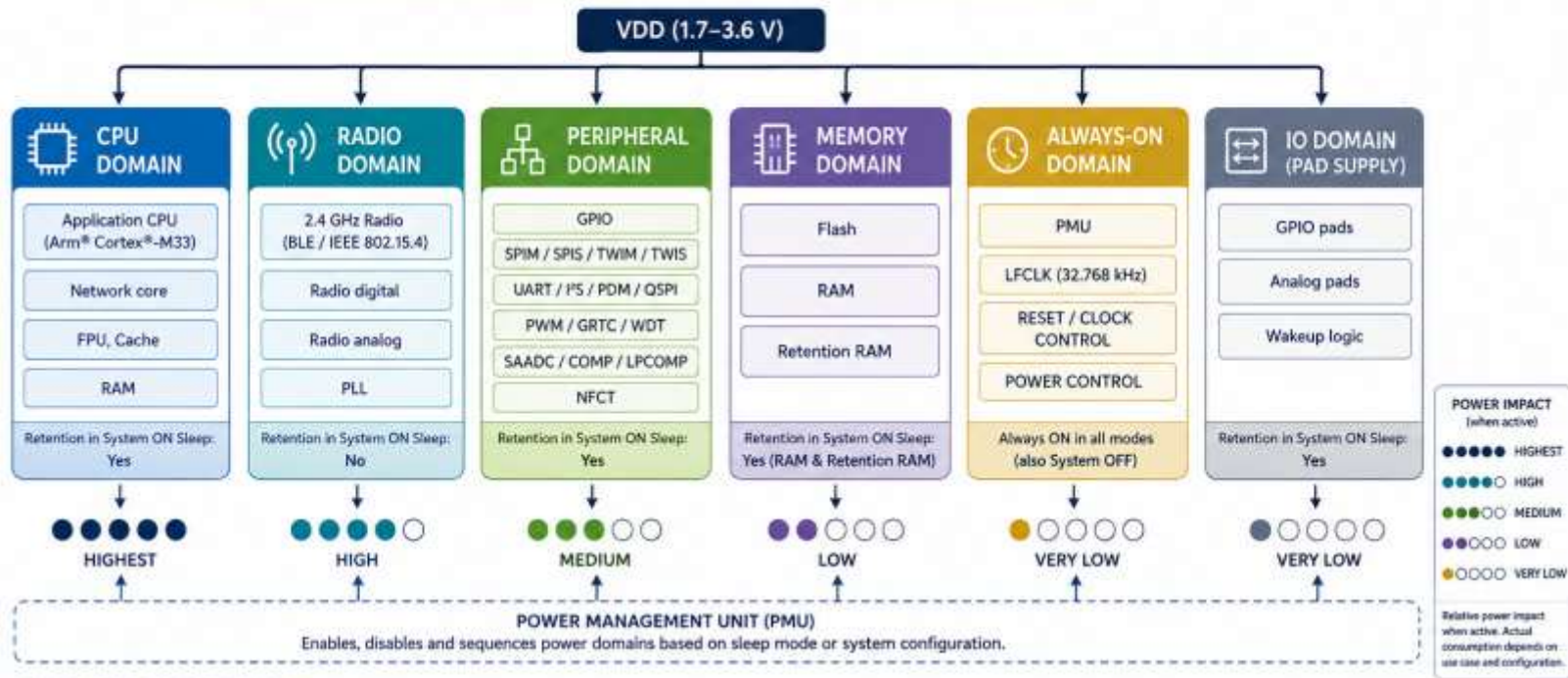
nRF54L15 / NCS / Zephyr's Power Management

What is supported by the Hardware?



nRF54L15 POWER DOMAIN OVERVIEW

Independent power domains that can be turned on/off to optimize power consumption



DOMAIN STATE IN SYSTEM MODES

MODE	CPU DOMAIN	RADIO DOMAIN	PERIPHERAL DOMAIN	MEMORY DOMAIN	ALWAYS-ON DOMAIN	IO DOMAIN (PAD SUPPLY)
ACTIVE	ON	ON (as needed)	ON (as needed)	ON	ON	ON
IDLE	CPU Sleep	ON (as needed)	ON (as needed)	ON	ON	ON
SYSTEM ON SLEEP	OFF	OFF	Optional (low power)	ON (RAM retained)	ON	ON
SYSTEM OFF	OFF	OFF	OFF	OFF (no retention)	ON	OFF



Most power is used by:
CPU Domain and Radio Domain
when active.



Save power by:
Disabling unused domains and
entering low-power sleep modes.

nRF54L Series Architecture: Power Domains

- Why are there Power Domains?

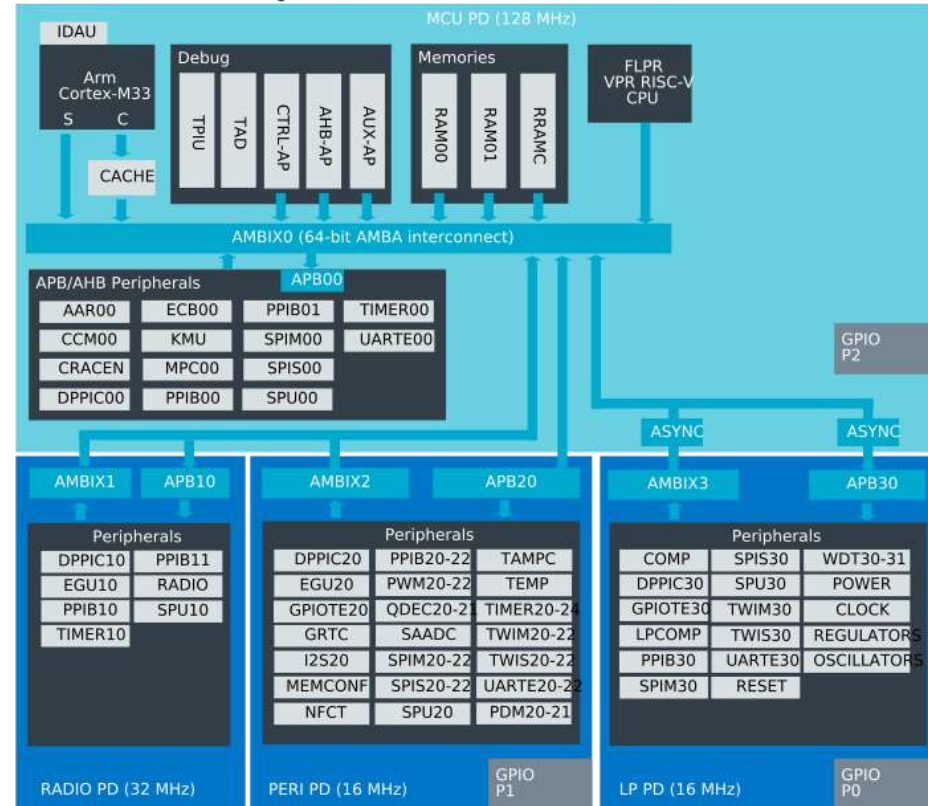
- Reduction of power consumption and leakage current
- Each power domain can be independently powered on or off

- How to use Power Domains?

- Hardware automatically manages power domains
- When no component in a domain is active, the domain is automatically shut down

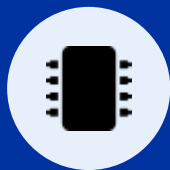
⇒ Group peripherals within same domain to minimize number of active domains

nRF54L15 Block Diagram



Two Independent Layers

NOT ON nRF54L15



System Power Management

`CONFIG_PM`

CPU-centric, in Zephyr generally.

The idle thread asks the policy manager which power state to enter, based on expected idle time and devicetree residency values.

AVAILABLE



Device Power Management

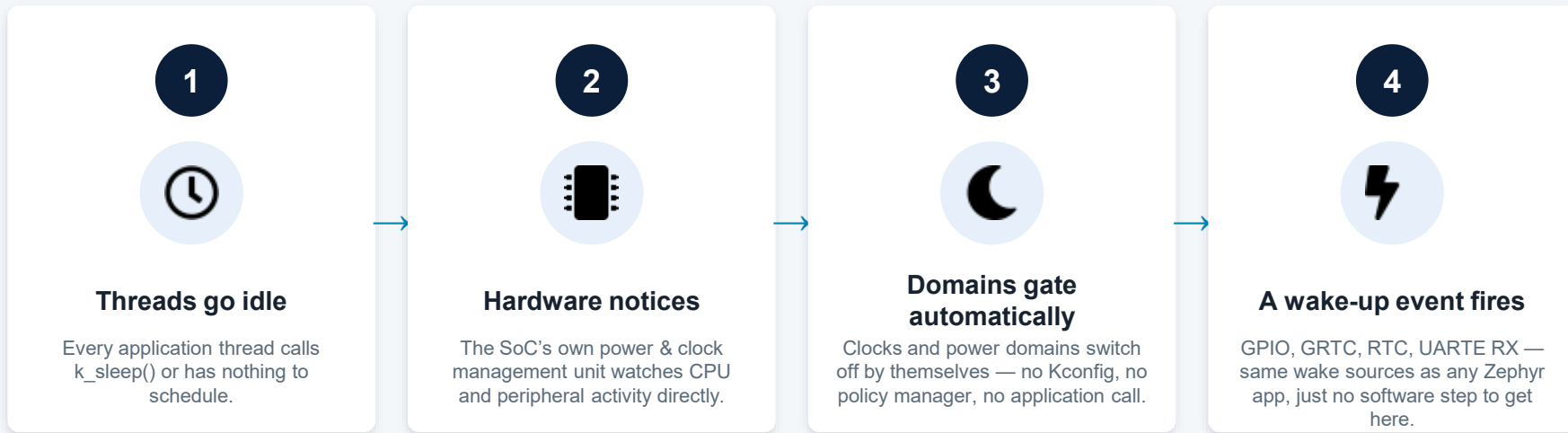
`CONFIG_PM_DEVICE / _RUNTIME`

Peripheral-centric.

Individual drivers (UART, SPI, I2C...) are suspended and resumed — actively, based on real usage, independent of CPU state.

On Nordic nRF54L SoCs – including nRF54L15 – only the device layer is software-configurable. Idle is automatic in hardware.

No idle thread decision — the SoC just does it



- There is no Zephyr Kconfig option that makes this happen and none that would stop it — it is a property of the silicon.
- The only things you still control: how much RAM stays retained, and which signals are allowed to wake the SoC from System OFF.

Two ways to power down a peripheral

	System-Managed PM	Device Runtime PM
Who decides?	The PM subsystem, passively	The driver / application, actively
Trigger	CPU about to enter a sleep state	Usage count reaches zero
Best for	Peripherals idle exactly when the CPU is idle	Bursty peripherals used while the CPU stays busy
Config	CONFIG_PM_DEVICE_SYSTEM_MANAGED	CONFIG_PM_DEVICE_RUNTIME

On the nRF54L15: System-Managed PM has no CPU-sleep transition to trigger it — use Device Runtime PM, the right-hand column, exclusively.

A Usage-Counted API

- A Device starts SUSPENDED

Until first requested – no wasted power at boot.

- `zephyr,pm-device-runtime-auto`

A single DeviceTree property enables this without touching driver init code.

- `pm_device_busy_set()/clear()`

Guards a transaction that must not be interrupted by a concurrent sleep decision.

```
pm_device_runtime_get(dev);

/* peripheral is guaranteed ACTIVE here */
perform_transaction(dev);

pm_device_runtime_put(dev);
/* suspended the moment nobody else
   holds a reference */
```


nRF54L15 Specifics

What the theory means on this particular SoC

What's different on this SoC



Global RTC (GRTC)

A shared low-power timekeeping and wake-up source across sleep states — different from the RTC/timer setup on nRF52-era parts. Stays active as a wake source by default, even if unused.



Multiple power domains

Independently switchable domains, managed automatically by hardware based on real-time activity — finer-grained than earlier nRF5-series SoCs.



CONFIG_PM cannot be selected

It depends on HAS_PM, which this SoC does not provide. There is no idle-state Kconfig here — by design, not because it's unfinished. Idle is 100% hardware-automatic.



System OFF is always explicit

There is no idle-thread path to System OFF at all on this SoC — reaching it always means calling `sys_poweroff()` from application code.

Code Pattern: Peripheral Device PM

CONFIG_PM_DEVICE lets a driver suspend/resume itself whenever the framework decides it's idle

❶ Enable it

CONFIG_PM_DEVICE=y in prj.conf

❷ Implement the hook

Handle PM_DEVICE_ACTION_SUSPEND/ _RESUME in your driver

❸ Register it

PM_DEVICE_DT_INST_DEFINE() ties the hook to the device instance

❹ Framework calls it for you

Runs automatically when the device is no longer needed

my_sensor.c

```
static int my_sensor_pm_action(const struct device *dev,
                               enum pm_device_action action)
{
    switch (action) {
        case PM_DEVICE_ACTION_SUSPEND:
            /* stop clocks, park sensor in standby */
            my_sensor_standby(dev);
            break;

        case PM_DEVICE_ACTION_RESUME:
            /* re-enable clocks, restore config */
            my_sensor_restore(dev);
            break;

        default:
            return -ENOTSUP;
    }
    return 0;
}

PM_DEVICE_DT_INST_DEFINE(0, my_sensor_pm_action);
SENSOR_DEVICE_DT_INST_DEFINE(0, my_sensor_init,
                             PM_DEVICE_DT_INST_GET(0),
                             &data, &config, POST_KERNEL,
                             CONFIG_SENSOR_INIT_PRIORITY, &api);
```

Keeping state alive through a full power-down

System OFF

Most RAM: powered OFF



Retained RAM region: kept powered



zephyr,retained-ram — a checksummed struct (boot count, uptime) survives the power cycle

- ✓ Wake sources must be configured explicitly — typically a GPIO with sense, or GRTC in its low-power mode.
- ✓ Retaining more RAM keeps more state — but also raises the residual System-OFF current. Size the region deliberately.
- ✓ A checksum lets your firmware tell "retained and valid" apart from "cold power-on, re-initialize".

Code Pattern: Entering System OFF

The deepest sleep state is never entered automatically – call it, and arrange your own wake-up

- CONFIG_POWEROFF=y in prj.conf
- Arm a wake source first (GRTC timer or GPIO) – without it, the device won't wake
- sys_poweroff() never returns; execution resumes at main() after a full reset
- Retained data must live in retained RAM or non-volatile storage – regular RAM is lost

main.c

```
#include <zephyr/drivers/timer/nrf_grtc_timer.h>
#include <zephyr/sys/poweroff.h>

#define SLEEP_TIME_S 2

void enter_system_off(void)
{
    int err = z_nrf_grtc_wakeup_prepare(
        SLEEP_TIME_S * USEC_PER_SEC);

    if (err) {
        return; /* wake source not armed, don't power off */
    }

    /* never returns – device wakes via reset, not resume */
    sys_poweroff();
}
```

What trips people up



Don't go looking for CONFIG_PM

It cannot be selected on this SoC — that's by design, not a missing Kconfig fragment. Idle is automatic in hardware.



Logging defeats measurement

An active UARTE skews every current reading — use RTT or disable logging when measuring.



The DK is not the product

Debug IC, LEDs, and other chips on the DK draw current a real PCB will not have.



Retention has a cost — in both states

Idle drops from ~2.9 μA (256 KB retained) to ~2.0 μA (96 KB) — size regions deliberately in Idle and System OFF alike.

Key Takeaways

Only Device PM is yours to configure

CONFIG_PM doesn't exist here — the SoC gates idle power in hardware, automatically.

Runtime PM for bursty peripherals

Suspend based on real usage, not CPU idle time — it works with or without system PM underneath.

System OFF is always explicit

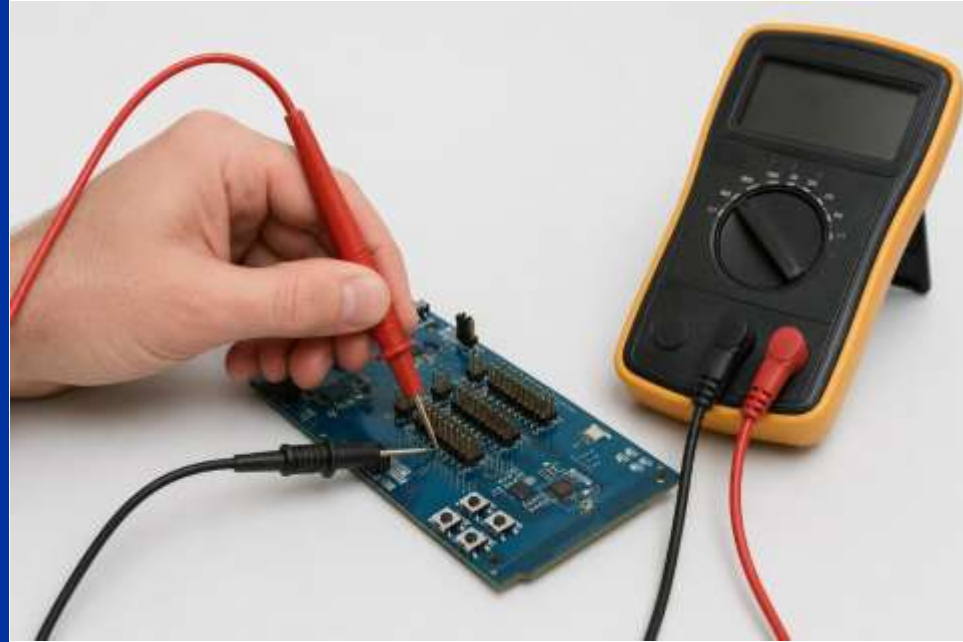
On the nRF54L15, call `sys_poweroff()` — there's no idle-thread path to it at all.

Measure, don't assume

A PPK2 trace is the only way to prove a power-management change actually worked.

Hands-on

- Power Management
 - Measuring an „unoptimized“ application
 - Idle states
 - Runtime PM on a Real Peripheral
 - System Off / RAM Retention





Step 1 – Create the sample

1. Download the lab project (1_baseline)
2. Open in Visual Studio Code
3. **Add build configuration** – use following settings:
 - **Board Target:** nrf54l15dk/nrf54l15/cpuapp
4. **Flash** the project



Step 2 - Check Kconfig Settings

5. Confirm default PM configuration. Use *nRF Kconfig GUI* tool.

Search for CONFIG_PM and CONFIG_PM_DEVICE.

Why can't you select the kconfig CONFIG_PM?



Step 3 – Measure the baseline current

6. Record the baseline current (sleep current) reading



PPK2

DSF64B2841A6



DATA LOGGER

SCOPE

ABOUT



POWER SUPPLY MODE:

Source meter **Ampere meter**

Enable power output



SAMPLING PARAMETERS

100,000 samples per second

Sample for **∞** forever**Start**Estimated space left: 995.14.310.46.22
88.7% Available disk space 1.8 TB

Save / Export

Screenshot

DISPLAY OPTIONS

Timestamps



Digital channels



0 1 2 3 4 5 6 7

Show Minimap



TEMP DISK

ADVANCED CONFIGURATION

SHOW DEE PANEL



CLEAR LOG

OPEN LOG FILE

AUTOCROLL LOG



SHOW LOG





Step 4 - Strip the application down to "nothing happening"

1. Create an extra Kconfig file with following content:

```
CONFIG_LOG=n  
CONFIG_SERIAL=n  
CONFIG_CONSOLE=n
```

2. Add the extra Kconfig to your build configuration



Step 5 - Measure the sleep current

3. Record the baseline current (sleep current) reading. Use a digital channel of PPK2 to check state of Led0.



PPK2

D5F64B2B41A6



DATA LOGGER

SCOPE

ABOUT



POWER SUPPLY MODE:

Source meter **Ampere meter**

Enable power output



SAMPLING PARAMETERS

100,000 samples per second

Sample for **∞** forever**Start**Estimated space used: 300 MB
88% Available disk space: 1.8 TB

Save / Export

Screenshot

DISPLAY OPTIONS

Timestamps



Digital channels

**0** 1 2 3 4 5 6 7

Show Minimap



TEMP DISK



ADVANCED CONFIGURATION

SHOW DEE PANEL



CLEAR LOG

OPEN LOG FILE

AUTOCROLL LOG



SHOW LOG



7

9



Step 6 – Check Device Runtime PM Sample

1. Download the lab project (2_device_pm)
2. Check prj.conf.
3. Check DeviceTree overlay file
4. Open main.c file and check static void send_demo_burst(void)
5. Measure current (use a digital channel on PPK2 to check P1.9 GPIO)



Step 8 – System OFF and RAM Retention

1. Please check following *Low Power State Sample for nrf5x* sample:

[samples/boards/nordic/system_off](#)